

Assignment 3 Report: Regret Matching for Multi-Agent Level-Based Foraging

Group 7

Name	Email
Bernardo Tavares Monteiro Fernandes Portugal	au804038@uni.au.dk
Mohamed Hashem Abdou	au656408@uni.au.dk
Rafael Ferreira da Costa Silva Valquaresma	au804039@uni.au.dk
Yu Jin	au785458@uni.au.dk

Problem Overview

We selected **Task B**: implement and study a MARL technique (Regret Matching) in the **Level-Based Foraging** world using **ir-sim**, scale the number of agents, and reflect on performance. We focus on:

- How regret-matching agents coordinate to collect rewards.
 - How collision avoidance and parameter choices affect efficiency and fairness.
 - How performance scales with **5, 10, and 20 agents**.
-

System Setup

- **Simulator:** ir-sim
 - **Environment:** 20×20 continuous world, fixed reward patches (subset selection for smaller agent counts)
 - **Robots:** differential-drive, 2D LIDAR
 - **Key files:**
 - **rm.py**: regret-matching logic, reward shaping, epsilon decay, global reward ledger, patch subset selection.
 - **collision_avoidance.py**: TTC-based speed limiting and escape turns.
 - **plot1.py**: CSV ingestion, mean/std plots, top-k agent comparisons.
 - **test.yaml**, **test_5.yaml**, **test_10.yaml**: configs for 20/5/10 agents.
 - **test.py**, **test_5.py**, **test_10.py**: experiment drivers with early-stop on full collection.
-

Method: Regret Matching (RM) + Collision Avoidance

- **State abstraction:** discretize (x, y) into grid cells (0.5 m).
 - **Actions (5):** forward, fwd+left, fwd+right, rotate left, rotate right.
 - **Policy:** sample actions positive regrets + -greedy exploration (decays from 0.30 $\rightarrow$ 0.05 over 800 steps).
 - **Rewards:**
 - (+) patch collection (once per patch, enforced by global ledger)
 - (–) step penalty
 - (+) shaping for reducing distance to nearest reward
 - (–) penalty for excessive rotation/standing still
 - **Collision avoidance:** TTC-based forward speed limiting; escape turn if $TTC < \text{threshold}$; hysteresis lock to keep turning until safe.
 - **Early termination:** stop episode when all rewards collected.
-

Experiments

Scenarios

- **Agent counts:** 5, 10, 20.
- **Reward subsets:**
 - 20 agents: full canonical list (17 patches).
 - 5 agents: 4 patches ($0.8 \times \text{agents}$, rounded) from the canonical list via `set_reward_patches`; random seed = 42; using random 4 indices of the canonical list.
 - 10 agents: 8 patches ($0.8 \times \text{agents}$, rounded) from the canonical list via `set_reward_patches`; random seed = 42; using random 8 indices of the canonical list.
- **Configs:** `vel_max` = [1.0, 2.0] for 5/10; `vel_max` = [1.0, 3.5] for 20; `COLLECT_RADIUS` = 0.4.

Metrics

- Episode return (per agent)
- Rewards collected (count, fairness)
- Steps to first/last collection; `since_last_collect`
- Fraction of rewards collected; time to full clearance
- Action distribution; regret statistics

Runs (representative, latest code)

- 5 agents: 4 patches; -decay on; `vel_max`=[1.0, 2.0].
- 10 agents: 8 patches; -decay on; `vel_max`=[1.0, 2.0].

- 20 agents: 17 patches; `vel_max=[1.0, 3.5]`.

(Exact CSVs: `rm_metrics_agent_*.csv`; plots in `plots/mean_std` and `plots/top5`.)

Results (Qualitative + Quantitative Highlights)

Overall

- **Faster convergence** after ledger/ -decay/rotation-penalty fixes.
- **Collision avoidance** keeps wasted steps low; agents “fan out” early.
- **Fairness improved** duplicate collections eliminated

5-Agent Results (0.8× patches from canonical list)

Results (5 agents)

Plots (5 agents)

- **Outcome:** All 4 patches collected; early stop at **step 2237** (Collected: 4/4).
- **Behavior:** Travel-heavy early phase; action mix ~30% forward, ~25% arcs, ~15% turns. No duplicate collections; TTC prevents stalls.
- **Takeaway:** Small team is travel-limited; slower decay or modest `vel_max` bump can reduce time-to-first-pickup.

10-Agent Results (0.8× patches from canonical list)

Results (10 agents)

Plots (10 agents)

- **Outcome:** All 8 patches collected; early stop at **step 1959** (Collected: 8/8).
- **Behavior:** 4/10 agents collected (2, 6, 7, 9); the others never picked up. Policy settles on forward/arcs; pure rotations ~7–11%.
- **Collections:** Few pickup events concentrated in those 4 agents; returns spike at pickups and stay negative for the rest. `dmin` drops near pickups then rises; non-collectors see `dmin` grow steadily.
- **Takeaway:** Exploration decays too fast (→0.05 by ~800 steps), freezing agents that didn’t find a patch early. Mitigate by prolonging/raising and/or removing the 0.8 patch scaling to increase collection signals.

20-Agent Results (17 patches, full list)

Results (20 agents)

Plots (20 agents)

- **Outcome:** All 17 patches collected; early stop at **step 1365** (Collected: 17/17).
- **Behavior:** Highest throughput; low `since_last_collect`; congestion managed by TTC + escape turns. Multiple agents collected (e.g., agents 1, 3, 4, 6, 8, 9, 11, 12, 14, 15 picked up at least one patch), while a few remained non-collectors but still contributed to coverage and blocking avoidance.
- **Takeaway:** Best overall speed; TTC keeps dense traffic flowing.

Scaling Summary

Agents	Patches used (rule)	Collected fraction	Steps to full collect	Notes
5	4 ($0.8 \times \text{agents}$)	1.00	2237	Travel-limited; completion achieved
10	8 ($0.8 \times \text{agents}$)	1.00	1959	Faster coverage; no congestion issues observed
20	17 (full list)	1.00	1365	Fastest overall; TTC mitigates congestion

Parameter Effects

- **-decay:** High initial `accelerates` discovery; too-early low `risks` stalls.

- **Vel_max / COLLECT_RADIUS:** Slightly higher values can help both dense and small teams; tune per scenario.

Behavioral Observations

- Agents self-assign “territories” without communication.
- Escape turns reduce deadlocks at intersections.
- With fewer agents (5), coverage holes appear unless `decay` stays higher for longer.

Why 5-Agents Can Feel Slower

- Lower natural coverage; each agent travels more.
- Fewer patches ($0.8 \times \text{agents}$) still give fewer shaping signals; if `decay` decays too early, distant patches stay unvisited.
-

Mitigations: slower `decay` for small teams, modest `vel_max` bump, or slightly fewer patches if needed.

Summary of the plots:

- The script ingests `rm_metrics_agent_*.csv` and produces figures for quick analysis: `mean ± std` across agents and top-5 curves per metric (returns, dmin, collected, regrets, action probabilities).
- Outputs are saved per run under `results_*/plots/mean_std/` and `results_*/plots/top5/`.

The 20-agent run gives the clearest visuals; link for reference:

Plots (20 agents)

Discussion

- RM works well without communication; collision avoidance plus shaping yields robust decentralized policies.
- `-decay` and `radius` sweeps expose clear trade-offs between speed, fairness, and total return.
- Scaling: congestion at 20 agents is manageable; at 5 agents, exploration dominates—tune `decay` and speed/radius accordingly.

- Limitations: no explicit coordination/role assignment; sensitivity to initial seeding; potential congestion on very tight maps.
 - Future work: heterogeneous per agent, lightweight signaling or region auctioning, adaptive state abstraction (clustering).
-

Conclusion

- **Regret Matching + TTC collision avoidance** delivers fast, fair, and scalable foraging in LBF.
 - Code fixes (global ledger), -decay, and shaping markedly improved convergence and fairness.
 - Systematic parameter sweeps (, agent count) provided insight and reproducibility for the assignment goals.
-

Artifacts

- **Code & data:** <https://github.com/Valquaresma03/MAS/tree/main>
- **Logs:** `rm_metrics_agent_*.csv` (per run and per agent)
- **Plots:** `plots/mean_std/`, `plots/top5/`
- **Configs:** `test.yaml`, `test_5.yaml`, `test_10.yaml`
- **Drivers:** `test.py`, `test_5.py`, `test_10.py`